

SOLUTION DESIGN DOCUMENT

Payroll Processing Automation

Scala 2.13+ | Apache Spark 3.5+ | Hadoop / HDFS

Document ID	SDD-PAYROLL-ETL-001
Version	1.0.0
Status	Draft
Author	Platform Engineering Team
Date	2026-05-25
Input	payroll.csv (~685,000 records, 18 columns)

Table of Contents

- 1. Introduction**
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Goals
 - 1.4 Assumptions
 - 1.5 Input Schema
- 2. System Architecture Overview**
 - 2.1 High-Level Architecture
 - 2.2 Package Structure
 - 2.3 Data Flow Diagram
 - 2.4 Runtime Environment
- 3. Data Ingestion and Validation Pipeline**
 - 3.1 Ingestion
 - 3.2 Validation
 - 3.3 Validation Rules Matrix
- 4. Transformation Logic**
 - 4.1 Current Transformations
 - 4.2 Derived Column Definitions
 - 4.3 Extended Transformations
 - 4.4 Pipeline Stages
- 5. Output Handling and Reporting**
 - 5.1 Output Artefacts
 - 5.2 Configuration
 - 5.3 Directory Layout
 - 5.4 Analytics Schema
- 6. Error Handling and Logging**
 - 6.1 Current Pattern
 - 6.2 Error Classification
 - 6.3 Structured Logging
 - 6.4 Winutils Flow
- 7. Performance Optimization**
 - 7.1 Schema Inference
 - 7.2 Partition Tuning
 - 7.3 Minimise Actions
 - 7.4 Caching
 - 7.5 SparkSession Config
- 8. Security and Compliance**
 - 8.1 Sensitive Fields
 - 8.2 Data Masking
 - 8.3 Path Security
 - 8.4 Compliance Checklist
- 9. Deployment and Configuration**
 - 9.1 Config Consolidation
 - 9.2 SBT Build
 - 9.3 Spark Submit
- 10. Testing Strategy**
 - 10.1 Unit — Transform
 - 10.2 Unit — Validation

10.3 Integration Test

11. Appendix

11.1 Glossary

11.2 Source File Index

11.3 Refactoring Priority

1. Introduction

1.1 Purpose

This document describes the solution design for automating payroll data processing using a Scala and Apache Spark-based ETL pipeline. It serves as the authoritative technical reference for engineering teams responsible for implementation, review, and maintenance of the system.

1.2 Scope

The pipeline processes **payroll.csv**, a city-level payroll dataset containing approximately **685,000+ records** across 18 columns. It covers ingestion, validation, transformation, analytics computation, and structured output delivery.

1.3 Goals

- Automate end-to-end payroll data processing with minimal manual intervention.
- Produce reliable, auditable outputs: department salary summaries and top-earner reports.
- Enforce data quality gates before any downstream transformation.
- Support local development on Windows (Winutils) and cluster-mode production deployment.

1.4 Assumptions and Constraints

- Input is a header-bearing CSV file with the schema defined in Section 1.5.
- Spark runs in local[*] mode for development; YARN/Kubernetes for production.
- Windows compatibility requires Winutils / Hadoop DLL bootstrapping (PayrollETL.setupWinutils).
- Output is written as partitioned CSVs to a local or HDFS-compatible path.
- Minimum supported versions: Scala 2.13+, Apache Spark 3.5+.

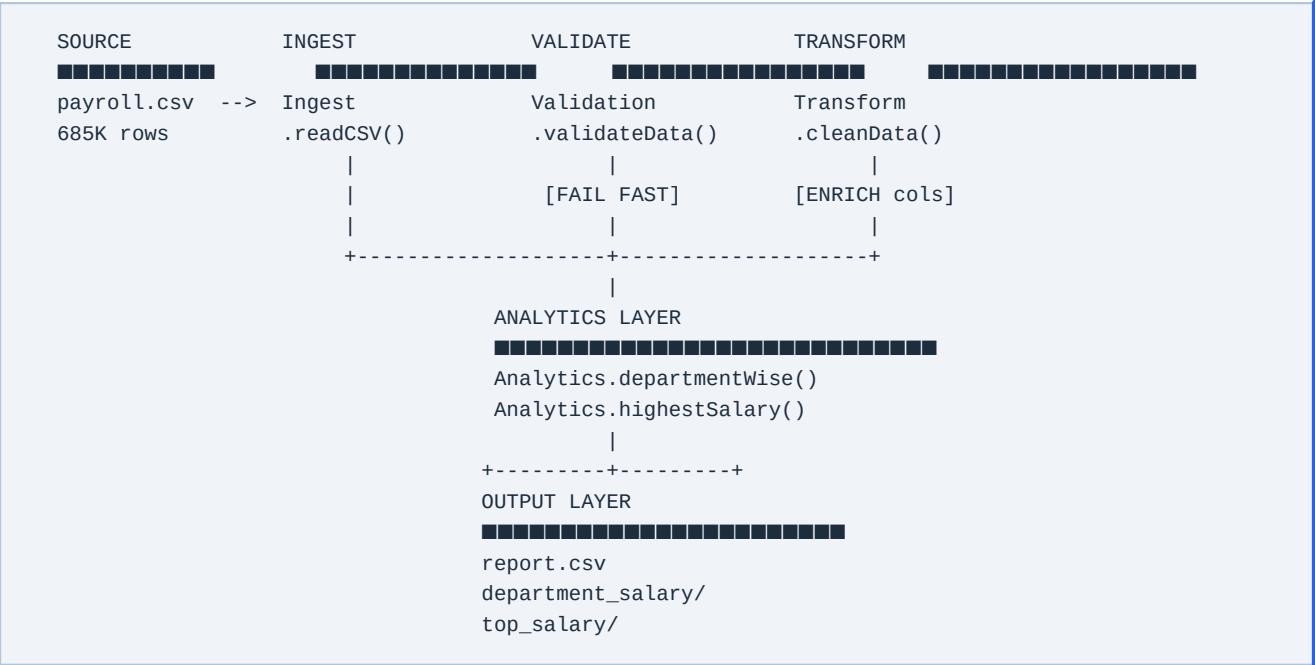
1.5 Input Schema — payroll.csv

Column	Type	Description
RECORD_NBR	String	Unique employee record identifier
PAY_YEAR	Integer	Payroll year
DEPARTMENT_NO	Integer	Numeric department code
DEPARTMENT_TITLE	String	Human-readable department name
JOB_CLASS_PGRADE	String	Job classification and pay grade code
JOB_TITLE	String	Employee job title
EMPLOYMENT_TYPE	String	FULL_TIME / PART_TIME
JOB_STATUS	String	ACTIVE / INACTIVE
MOU	String	Memorandum of Understanding code

Column	Type	Description
MOU_TITLE	String	MOU description
REGULAR_PAY	Double	Base regular pay amount
OVERTIME_PAY	Double	Overtime pay amount
ALL_OTHER_PAY	Double	Miscellaneous pay components
TOTAL_PAY	Double	Sum of all pay components
CITY_RETIREMENT_CONTRIBUTIONS	Double	Employer retirement contribution
BENEFIT_PAY	Double	Employer benefits contribution
GENDER	String	Gender designation
ETHNICITY	String	Ethnicity designation

2. System Architecture Overview

2.1 High-Level Architecture



2.2 Package Structure

```
com.payroll
|-- main
|   |-- PayrollETL      <- Orchestration entry point
|-- Config
|   |-- Config          <- Centralised paths & parameters
|-- ingestion
|   |-- package         <- Package object (utilities placeholder)
|   |-- Ingest          <- CSV reader abstraction
|-- validation
|   |-- Validation      <- Null count & duplicate detection
|-- transformation
|   |-- Transform       <- Cleaning and column derivation
+-- analytics
    |-- Analytics       <- Aggregation and ranking
```

2.3 Data Flow Diagram

```
payroll.csv (685K+ rows x 18 cols)
|
v  spark.read.option("header","true")
+-----+ .option("inferSchema","true").csv(path)
| INGESTION LAYER |
| Ingest.readCSV() | --> rawDF [18 cols]
+-----+
|
v  per-column null counts + duplicate detection
+-----+
| VALIDATION LAYER | --> quality report (fail on threshold breach)
| .validateData() |
+-----+
| rawDF (non-destructive pass-through)
v
+-----+
| TRANSFORMATION LAYER |
| 1. na.drop()          remove null rows |
| 2. dropDuplicates()   remove exact dups |
| 3. + Annual_Salary    REGULAR_PAY x 12 |
| 4. + Bonus            REGULAR_PAY x 0.1 |
+-----+
| transformedDF [20 cols]
v
+-----+
| ANALYTICS LAYER |
| departmentWiseSalary() |
|   groupBy + avg/sum Annual_Salary |
| highestSalary() |
|   orderBy desc + limit(10) |
+-----+
|
v
+-----+
| OUTPUT LAYER |
| /Output/report.csv <- cleaned data |
| /Output/dept_salary/ <- aggregated |
| /Output/top_salary/ <- top 10 earners|
+-----+
```

2.4 Runtime Environment

Environment	Spark Master	Hadoop	Notes
Development	local[*]	Winutils 3.3.5	Windows, IntelliJ IDEA
Staging	local[4]	HDFS optional	Linux/Mac, limited cores
Production	yarn / k8s	HDFS / S3	Cluster mode, external config

3. Data Ingestion and Validation Pipeline

3.1 Ingestion

The **Ingest** object provides a single reusable CSV reader. `inferSchema=true` interrogates the data to assign appropriate types. In production, replace with an explicit `StructType` to eliminate the two-pass scan and prevent silent type mismatches on schema drift.

```
// com.payroll.ingestion.Ingest (current implementation)
object Ingest {
  def readCSV(spark: SparkSession, path: String): DataFrame =
    spark.read
      .option("header", "true")
      .option("inferSchema", "true")
      .csv(path)
}

// Production recommendation -- explicit StructType (no extra scan pass)
val payrollSchema = StructType(Seq(
  StructField("RECORD_NBR", StringType, nullable = true),
  StructField("PAY_YEAR", IntegerType, nullable = true),
  StructField("DEPARTMENT_NO", IntegerType, nullable = true),
  StructField("DEPARTMENT_TITLE", StringType, nullable = true),
  StructField("JOB_CLASS_PGRADE", StringType, nullable = true),
  StructField("JOB_TITLE", StringType, nullable = true),
  StructField("EMPLOYMENT_TYPE", StringType, nullable = true),
  StructField("JOB_STATUS", StringType, nullable = true),
  StructField("MOU", StringType, nullable = true),
  StructField("MOU_TITLE", StringType, nullable = true),
  StructField("REGULAR_PAY", DoubleType, nullable = true),
  StructField("OVERTIME_PAY", DoubleType, nullable = true),
  StructField("ALL_OTHER_PAY", DoubleType, nullable = true),
  StructField("TOTAL_PAY", DoubleType, nullable = true),
  StructField("CITY_RETIREMENT_CONTRIBUTIONS", DoubleType, nullable = true),
  StructField("BENEFIT_PAY", DoubleType, nullable = true),
  StructField("GENDER", StringType, nullable = true),
  StructField("ETHNICITY", StringType, nullable = true)
))

spark.read
  .option("header", "true")
  .option("mode", "PERMISSIVE")
  .option("columnNameOfCorruptRecord", "_corrupt_record")
  .schema(payrollSchema)
  .csv(path)
```

3.2 Validation

The current Validation object prints null counts per column and total duplicate rows to stdout. The recommended enhancement adds configurable thresholds and throws `IllegalStateException` on critical column failures to halt the pipeline before corrupt data enters transformation.


```
// Current implementation (com.payroll.validation.Validation)
object Validation {
  def validateData(df: DataFrame): Unit = {
    println("===== NULL COUNT =====")
    df.columns.foreach { column =>
      val nullCount = df.filter(col(column).isNull).count()
      println(s"$column -> $nullCount")
    }
    println("===== DUPLICATE COUNT =====")
    val duplicateCount = df.count() - df.dropDuplicates().count()
    println(s"Duplicate Rows: $duplicateCount")
  }
}

// Enhanced: single-pass aggregation + threshold enforcement
object Validation {
  val NULL_THRESHOLD_PCT = 5.0
  val criticalColumns    = Set("RECORD_NBR", "REGULAR_PAY", "DEPARTMENT_TITLE")

  def validateData(df: DataFrame): Unit = {
    val total = df.count()
    // Single pass: count nulls for ALL columns in one action
    val nullCounts = df.select(
      df.columns.map(c => sum(col(c).isNull.cast("int")).alias(c)): _*
    ).first()

    df.columns.foreach { column =>
      val nullCount = nullCounts.getAs[Long](column)
      val pct       = (nullCount.toDouble / total) * 100
      if (criticalColumns.contains(column) && pct > NULL_THRESHOLD_PCT)
        throw new IllegalStateException(
          s"Validation FAILED: $column has ${pct}% nulls (threshold $NULL_THRESHOLD_PCT%)"
        )
    }
    val dupes = total - df.dropDuplicates().count()
    if (dupes > 0) println(s"WARNING: $dupes duplicate rows detected")
  }
}
```

3.3 Validation Rules Matrix

Rule ID	Column(s)	Check	Severity	On Failure
VR-001	RECORD_NBR	Not null, not empty	CRITICAL	Abort pipeline
VR-002	REGULAR_PAY	Not null, >= 0.0	CRITICAL	Abort pipeline
VR-003	DEPARTMENT_TITLE	Not null, not empty	CRITICAL	Abort pipeline
VR-004	TOTAL_PAY	>= REGULAR_PAY	WARNING	Log and continue
VR-005	EMPLOYMENT_TYPE	In {FULL_TIME, PART_TIME}	WARNING	Log and continue
VR-006	All rows	No exact duplicates	WARNING	Log count
VR-007	PAY_YEAR	Between 2000 and current year	WARNING	Log and continue

4. Transformation Logic for Payroll Calculations

4.1 Current Transformations

```
// com.payroll.transformation.Transform
object Transform {
  def cleanData(df: DataFrame): DataFrame = {

    val cleanedDF = df.na.drop()      // remove rows with ANY null
    .dropDuplicates()                // remove exact duplicate rows

    val transformedDF = cleanedDF
    .withColumn("Annual_Salary", col("REGULAR_PAY") * 12)
    .withColumn("Bonus",          col("REGULAR_PAY") * 0.10)

    transformedDF
  }
}
```

4.2 Derived Column Definitions

Derived Column	Formula	Business Meaning
Annual_Salary	REGULAR_PAY * 12	Annualised base compensation estimate
Bonus	REGULAR_PAY * 0.10	Standard 10% performance bonus estimate
Total_Compensation	Annual_Salary + OT + Other + Benefits + Retirement	Full employer cost per employee
Overtime_Ratio	OVERTIME_PAY / REGULAR_PAY	Proportion of OT vs base pay
Benefits_To_Base_Ratio	BENEFIT_PAY / REGULAR_PAY	Benefits cost as share of base pay
Pay_Grade	Band on Annual_Salary	GRADE_1 < \$50K ... GRADE_4 > \$150K

4.3 Extended Transformation Recommendations

```
object Transform {  
  def cleanData(df: DataFrame): DataFrame = {  
  
    // Stage 1: null-drop on key columns only (preserve optional-null rows)  
    val cleanedDF = df  
      .na.drop(cols = Seq("RECORD_NBR", "REGULAR_PAY", "DEPARTMENT_TITLE"))  
      .dropDuplicates(Seq("RECORD_NBR", "PAY_YEAR")) // key-based dedup  
  
    // Stage 2: fill optional pay fields with 0.0  
    val defaultedDF = cleanedDF.na.fill(0.0,  
      Seq("OVERTIME_PAY", "ALL_OTHER_PAY",  
        "CITY_RETIREMENT_CONTRIBUTIONS", "BENEFIT_PAY"))  
  
    // Stage 3: derived columns  
    defaultedDF  
      .withColumn("Annual_Salary",  
        col("REGULAR_PAY") * 12)  
      .withColumn("Bonus",  
        col("REGULAR_PAY") * 0.10)  
      .withColumn("Total_Compensation",  
        col("Annual_Salary") + col("OVERTIME_PAY") + col("ALL_OTHER_PAY")  
        + col("BENEFIT_PAY") + col("CITY_RETIREMENT_CONTRIBUTIONS"))  
      .withColumn("Overtime_Ratio",  
        when(col("REGULAR_PAY") > 0,  
          col("OVERTIME_PAY") / col("REGULAR_PAY"))  
        .otherwise(lit(0.0)))  
      .withColumn("Pay_Grade",  
        when(col("Annual_Salary") < 50000, lit("GRADE_1"))  
        .when(col("Annual_Salary") < 100000, lit("GRADE_2"))  
        .when(col("Annual_Salary") < 150000, lit("GRADE_3"))  
        .otherwise(lit("GRADE_4")))  
  }  
}
```

4.4 Transformation Pipeline Stages

```
rawDF (685K rows x 18 cols)  
|  
v Stage 1: NULL DROP on key columns (RECORD_NBR, REGULAR_PAY, DEPT_TITLE)  
v Stage 2: DEDUPLICATION on composite key (RECORD_NBR, PAY_YEAR)  
v Stage 3: NUMERIC DEFAULT FILL -- 0.0 for optional pay/benefit cols  
v Stage 4: COLUMN DERIVATION -- 6 new columns appended  
|  
transformedDF (<= 685K rows x 24 cols)
```

5. Output Handling and Reporting

5.1 Output Artefacts

Output File	Source DataFrame	Format	Mode	Description
report.csv	transformedDF	CSV	Overwrite	Full cleaned & enriched payroll dataset
department_salary/	deptSalaryDF	CSV	Overwrite	Avg and total Annual_Salary per department
top_salary/	topSalaryDF	CSV	Overwrite	Top 10 highest Annual_Salary records

5.2 Output Configuration

```
// From PayrollETL.scala

// Full cleaned data
transformedDF.write
  .mode("overwrite")
  .option("header", "true")
  .csv(Config.processedPath)

// Department-level analytics
deptSalaryDF.write
  .mode("overwrite")
  .option("header", "true")
  .csv(Config.outputPath + "department_salary")

// Top 10 earners
topSalaryDF.write
  .mode("overwrite")
  .option("header", "true")
  .csv(Config.outputPath + "top_salary")
```

5.3 Output Directory Layout

```
/Output/
|-- report.csv/
|   |-- _SUCCESS
|   +-- part-00000-*.csv
|-- department_salary/
|   |-- _SUCCESS
|   +-- part-00000-*.csv
+-- top_salary/
    |-- _SUCCESS
    +-- part-00000-*.csv
```

■ Spark writes partitioned part-files by default. For single-file delivery (e.g. SFTP), call `.coalesce(1)` before `.write`, then rename the `part-*.csv` file.

5.4 Analytics Schema

Department Salary (deptSalaryDF)

```
df.groupBy("DEPARTMENT_TITLE")
  .agg(
    avg("Annual_Salary").alias("Average_Salary"),
    sum("Annual_Salary").alias("Total_Salary")
  )
-- Output columns: DEPARTMENT_TITLE | Average_Salary | Total_Salary
```

Top 10 Salaries (topSalaryDF)

```
df.orderBy(col("Annual_Salary").desc).limit(10)
-- Output: full 20-column row for each of the 10 highest Annual_Salary values
```

6. Error Handling and Logging

6.1 Current Pattern

PayrollETL.main() wraps the entire pipeline in a try-catch block that walks the full cause chain and re-throws to propagate a non-zero exit code to the Spark scheduler.

```
try {
  // ... pipeline stages ...
} catch {
  case e: Throwable =>
    System.err.println("ETL Pipeline failed with exception:")
    e.printStackTrace()
    var cause = e.getCause
    while (cause != null) {
      System.err.println("Caused by:")
      cause.printStackTrace()
      cause = cause.getCause
    }
    throw e // re-throw: non-zero exit propagates to scheduler
}
```

6.2 Error Classification

Category	Examples	Current Handling	Recommended Action
Infrastructure	Hadoop DLL missing, Winutils unavailable	setupWinutils()	Log warning, attempt fallback
Ingestion	File not found, malformed CSV row	Caught in main	Log path, abort cleanly
Validation	Critical column null rate > threshold	println only	Throw IllegalStateException
Transformation	Column not found, type mismatch	Caught in main	Log column name, abort
Output	Directory permissions, disk space	Caught in main	Log path, alert ops
Spark	Executor OOM, task failures	Re-thrown	Configure retry + alerting

6.3 Recommended Structured Logging

```
// Replace println with SLF4J (bundled with Spark)
import org.slf4j.LoggerFactory
private val logger = LoggerFactory.getLogger(getClass)

logger.info("PayrollETL pipeline starting")
logger.info(s"Ingested ${rawDF.count()} records from ${Config.rawPath}")
logger.info("Validation passed")
logger.info(s"Transformation complete: ${transformedDF.count()} records retained")
logger.info("PayrollETL pipeline completed successfully")
logger.error("PayrollETL pipeline failed", e) // includes stack trace
```

6.4 Windows Winutils Bootstrap Flow

```
setupWinutils()  
|  
+-- Is Windows? --- No --> Skip (no-op on Linux/Mac)  
|  
Yes  
|  
+-- Download winutils.exe from GitHub cdarlint/winutils  
|   Success --> Set System.setProperty("hadoop.home.dir", ...)  
|   Failure --> Copy hostname.exe as dummy executable  
|               Failure --> createNewFile() (empty stub, non-fatal)  
|  
+-- Download hadoop.dll from GitHub  
|   Success --> System.load(hadoopDllPath)  
|   Failure --> Log warning, continue (non-fatal)  
|  
+-- hadoop.home.dir set --> SparkSession.builder() proceeds normally
```

7. Performance Optimization Strategies

7.1 Schema Inference vs. Explicit Schema

Approach	Scan Cost	Type Safety	Recommended Use
inferSchema=true	2 full scans	Low	Development only
Explicit StructType	0 extra scans	High	Staging + Production

Supplying an explicit schema eliminates the preliminary scan, saving one full read of the 685K-row file on every pipeline execution.

7.2 Partition Tuning

```
// Default shuffle partitions (200) are excessive for local mode
spark.conf.set("spark.sql.shuffle.partitions", "8") // set to 2x CPU cores

// Adaptive Query Execution (Spark 3.x) -- auto-optimises joins and shuffles
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

7.3 Minimise Action Calls

Each `.count()` triggers a full `DataFrame` scan. Current Validation fires one action per column plus two more for duplicate detection. Replace with a single-pass aggregation:

```
// BAD: N actions (one count() per column)
df.columns.foreach { c =>
  val nullCount = df.filter(col(c).isNull).count()
}

// GOOD: 1 action for all columns combined
val nullCounts = df.select(
  df.columns.map(c => sum(col(c).isNull.cast("int")).alias(c)): _*
).first()
// Reduction: N+2 actions --> 1 action
```

7.4 Caching Strategy

```
// Cache rawDF -- it is consumed by BOTH validateData() AND cleanData()
rawDF.cache()

Validation.validateData(rawDF) // action 1 -- hits cache
val transformedDF = Transform.cleanData(rawDF) // action 2 -- hits cache

rawDF.unpersist() // release memory after last use
```

7.5 Full SparkSession Configuration


```
val spark = SparkSession.builder()
  .appName("Payroll ETL Project")
  .master("local[*]")
  .config("spark.sql.shuffle.partitions", "8")
  .config("spark.sql.adaptive.enabled", "true")
  .config("spark.sql.adaptive.coalescePartitions.enabled", "true")
  .config("spark.driver.memory", "4g")
  .config("spark.executor.memory", "4g")
  .getOrCreate()
```

8. Security and Compliance Considerations

8.1 Sensitive Fields in payroll.csv

Field	Sensitivity	Risk
RECORD_NBR	PII (identifier)	Employee re-identification
GENDER	Protected class	Discriminatory analytics exposure
ETHNICITY	Protected class	Discriminatory analytics exposure
REGULAR_PAY	Confidential	Compensation disclosure
TOTAL_PAY	Confidential	Compensation disclosure

8.2 Data Masking and Pseudonymisation

```
object DataMasker {  
  
  // One-way hash of RECORD_NBR: preserves join-ability, removes direct PII  
  def maskRecordNbr(df: DataFrame): DataFrame =  
    df.withColumn("RECORD_NBR", sha2(col("RECORD_NBR"), 256))  
  
  // Suppress protected-class columns for non-privileged reporting  
  def redactProtectedFields(df: DataFrame): DataFrame =  
    df.drop("GENDER", "ETHNICITY")  
  
  // Salary banding instead of exact figures for general distribution  
  def bandSalary(df: DataFrame): DataFrame =  
    df.withColumn("Salary_Band",  
      when(col("Annual_Salary") < 60000, lit("< $60K"))  
      .when(col("Annual_Salary") < 100000, lit("$60K-$100K"))  
      .when(col("Annual_Salary") < 150000, lit("$100K-$150K"))  
      .otherwise(lit("> $150K"))  
    ).drop("REGULAR_PAY", "Annual_Salary", "TOTAL_PAY")  
}
```

8.3 File and Path Security

- Config.rawPath should be resolved at runtime from environment variables or a secrets manager — never hardcoded in source.
- Output directories should be writable only by the service account running Spark.
- Avoid logging salary or PII values via println or structured log statements.

```
// Resolve sensitive paths from environment at runtime  
object Config {  
  val rawPath: String =  
    sys.env.getOrElse("PAYROLL_RAW_PATH", "fallback/path/payroll.csv")  
  val processedPath: String =  
    sys.env.getOrElse("PAYROLL_PROCESSED_PATH", "fallback/path/report.csv")  
  val outputPath: String =  
    sys.env.getOrElse("PAYROLL_OUTPUT_PATH", "fallback/path/output/")  
}
```

8.4 Compliance Checklist

Requirement	Control	Status
PII not logged to console	Avoid df.show() in production	Implement
Sensitive columns masked	DataMasker.redactProtectedFields()	Implement
Output access control	OS-level permissions on output directory	Implement
Audit trail of pipeline runs	Structured logging with run IDs	Implement
Hardcoded paths removed	Environment variable resolution	Implement
Protected-class field access ctrl	Role-based output separation	Implement

9. Deployment and Configuration

9.1 Configuration Consolidation

Three standalone path objects (`rawPath.scala`, `processedPath.scala`, `outputPath.scala`) exist alongside `Config.scala`. These are redundant and risk diverging. Eliminate them in favour of a single consolidated `Config` object:

```
// Recommended: single Config object
object Config {
  val rawPath:      String = sys.env.getOrElse("PAYROLL_RAW_PATH",      "...")
  val processedPath: String = sys.env.getOrElse("PAYROLL_PROCESSED_PATH", "...")
  val outputPath:   String = sys.env.getOrElse("PAYROLL_OUTPUT_PATH",   "...")

  // Tunable parameters
  val maxNullPctCritical: Double = 5.0
  val bonusRate:         Double = 0.10
  val shufflePartitions: Int     = 8
  val driverMemory:      String = "4g"
  val executorMemory:    String = "4g"
}
```

9.2 Build Tool — SBT

```
name      := "PayrollETL"
version   := "1.0.0"
scalaVersion := "2.13.12"

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-core"      % "3.5.0" % "provided",
  "org.apache.spark" %% "spark-sql"       % "3.5.0" % "provided",
  "org.slf4j"         % "slf4j-api"       % "2.0.9",
  "ch.qos.logback"    % "logback-classic" % "1.4.14" % "runtime",
  "org.scalatest"     %% "scalatest"      % "3.2.17" % "test"
)
```

9.3 Spark Submit — Production

```
spark-submit \
  --class com.payroll.main.PayrollETL \
  --master yarn \
  --deploy-mode cluster \
  --driver-memory 4g \
  --executor-memory 4g \
  --executor-cores 4 \
  --num-executors 3 \
  --conf spark.sql.shuffle.partitions=32 \
  --conf spark.sql.adaptive.enabled=true \
  payroll-etl-assembly-1.0.0.jar
```

10. Testing Strategy

10.1 Unit Test — Transform

```
class TransformSpec extends AnyFlatSpec with Matchers {  
  val spark = SparkSession.builder().master("local").appName("test").getOrCreate()  
  import spark.implicits._  
  
  it should "derive Annual_Salary as REGULAR_PAY x 12" in {  
    val input = Seq((1, 5000.0)).toDF("RECORD_NBR", "REGULAR_PAY")  
    Transform.cleanData(input)  
      .select("Annual_Salary").first().getDouble(0) shouldEqual 60000.0  
  }  
  
  it should "drop rows with null REGULAR_PAY" in {  
    val input = Seq((1, Some(5000.0)), (2, None))  
      .toDF("RECORD_NBR", "REGULAR_PAY")  
    Transform.cleanData(input).count() shouldEqual 1  
  }  
}
```

10.2 Unit Test — Validation

```
class ValidationSpec extends AnyFlatSpec with Matchers {  
  it should "throw IllegalStateException when critical column null rate exceeds 5%" in {  
    val input = Seq.fill(100)(None: Option[String])  
      .map(r => Tuple1(r)).toDF("RECORD_NBR")  
    a [IllegalStateException] should be thrownBy Validation.validateData(input)  
  }  
}
```

10.3 Integration Test

An integration test should:

- Copy a small slice of payroll.csv (e.g. 1,000 rows) to a temporary directory.
- Execute PayrollETL.main() against it using test-specific input/output paths.
- Assert output files exist, contain a _SUCCESS marker, and have expected row counts.
- Verify derived columns (Annual_Salary, Bonus) are numerically correct on sample rows.

11. Appendix

11.1 Glossary

Term	Definition
ETL	Extract, Transform, Load — the pattern for data pipeline design
DataFrame	Spark's distributed, schema-aware tabular data structure
Action	A Spark operation that triggers computation, e.g. <code>.count()</code> , <code>.show()</code>
Transformation	A lazy Spark operation that builds the execution plan without executing
inferSchema	Spark's two-pass mechanism to auto-detect column types from CSV content
Winutils	Windows compatibility binary required by Spark for Hadoop I/O operations
MOU	Memorandum of Understanding — a labour agreement grouping
AQE	Adaptive Query Execution — Spark 3.x runtime query plan optimisation

11.2 Source File Index

File	Package	Responsibility
PayrollETL.scala	com.payroll.main	Orchestration entry point
Config.scala	com.payroll.Config	Centralised path and parameter config
Ingest.scala	com.payroll.ingestion	CSV reader abstraction over SparkSession
Validation.scala	com.payroll.validation	Data quality checks (null count, dupes)
Transform.scala	com.payroll.transformation	Cleaning and column derivation
Analytics.scala	com.payroll.analytics	Aggregation and ranking computations
package.scala	com.payroll.ingestion	Package object (utilities placeholder)
rawPath.scala	com.payroll.Config	REDUNDANT -- consolidate into Config
processedPath.scala	com.payroll.Config	REDUNDANT -- consolidate into Config
outputPath.scala	com.payroll.Config	REDUNDANT -- consolidate into Config

11.3 Recommended Refactoring Priority

High	Consolidate rawPath / processedPath / outputPath into Config
High	Replace inferSchema=true with explicit StructType
High	Upgrade Validation to throw on critical column failures
Medium	Replace println with SLF4J structured logger

Medium	Mask PII fields before any show() or output write
Medium	Add spark.sql.adaptive.enabled=true to SparkSession config
Low	Add ScalaTest unit tests for Transform and Validation
Low	Parameterise bonusRate and shufflePartitions via Config

End of Document — SDD-PAYROLL-ETL-001 v1.0.0 — Platform Engineering Team — 2026-05-25